Lab Programs:

7- Implement different types of Joins: Implement various types of joins in SQL, including Inner Join, Outer Join, and Natural Join, with suitable examples.

INPUT:

```
CREATE TABLE departments (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(50)
);
CREATE TABLE employees (
emp_id INT PRIMARY KEY,
emp_name VARCHAR(50),
dept_id INT,
salary INT
);
INSERT INTO departments VALUES
(1,'HR'),(2,'IT'),(3,'Finance'),(4,'Marketing');
INSERT INTO employees VALUES
(101,'Asha',1,30000),(102,'Ramya',2,50000),(103,'Mehur',2,55000),
(104,'Kiran',3,45000),(105,'Niiha',NULL,40000);
SELECT e.emp_name, d.dept_name
FROM employees e INNER JOIN departments d ON e.dept_id = d.dept_id;
SELECT e.emp_name, d.dept_name
FROM employees e LEFT JOIN departments d ON e.dept_id = d.dept_id;
SELECT e.emp_name, d.dept_name
FROM employees e RIGHT JOIN departments d ON e.dept_id = d.dept_id;
SELECT e.emp_name, d.dept_name
FROM employees e LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_name, d.dept_name
FROM employees e RIGHT JOIN departments d ON e.dept_id = d.dept_id;
SELECT * FROM employees NATURAL JOIN departments;
```

OUTPUT:

Output:

```
+----------+-----------+
| emp_name | dept_name |
+----------+-----------+
| Asha     | HR        |
| Ramya    | IT        |
| Mehur    | IT        |
| Kiran    | Finance   |
+----------+-----------+
+----------+-----------+
| emp_name | dept_name |
+----------+-----------+
| Asha     | HR        |
| Ramya    | IT        |
| Mehur    | IT        |
| Kiran    | Finance   |
| Niiha    | NULL      |
+----------+-----------+
+----------+-----------+
| emp_name | dept_name |
+----------+-----------+
| Asha     | HR        |
| Mehur    | IT        |
| Ramya    | IT        |
| Kiran    | Finance   |
| NULL     | Marketing |
+----------+-----------+
+----------+-----------+
| emp_name | dept_name |
+----------+-----------+
| Asha     | HR        |
| Ramya    | IT        |
| Mehur    | IT        |
| Kiran    | Finance   |
| Niiha    | NULL      |
| NULL     | Marketing |
+----------+-----------+
+---------+--------+----------+--------+-----------+
| dept_id | emp_id | emp_name | salary | dept_name |
+---------+--------+----------+--------+-----------+
|       1 |    101 | Asha     |  30000 | HR        |
|       2 |    102 | Ramya    |  50000 | IT        |
|       2 |    103 | Mehur    |  55000 | IT        |
|       3 |    104 | Kiran    |  45000 | Finance   |
+---------+--------+----------+--------+-----------+
```

8- Implement Grouping and Ordering: Study and implement the GROUP BY and HAVING clauses, ORDER BY clause.

INPUT:

```
CREATE TABLE departments (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(50)
);
CREATE TABLE employees (
emp_id INT PRIMARY KEY,
emp_name VARCHAR(50),
dept_id INT,
salary INT
);
INSERT INTO departments VALUES
(1,'HR'),(2,'IT'),(3,'Finance'),(4,'Marketing');
INSERT INTO employees VALUES
(101,'Asha',1,30000),(102,'Ravi',2,50000),(103,'Meera',2,55000),
(104,'Kiran',3,45000),(105,'Neha',NULL,40000);
SELECT dept_id, COUNT(*) AS total_emp, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id;
SELECT dept_id, COUNT(*) AS total_emp
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 1;
SELECT emp_name, salary FROM employees ORDER BY salary DESC;
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
WHERE dept_id IS NOT NULL
GROUP BY dept_id
HAVING AVG(salary) > 40000
ORDER BY avg_salary DESC;
```

OUTPUT:

Output:

```
+---------+-----------+------------+
| dept_id | total_emp | avg_salary |
+---------+-----------+------------+
|       1 |         1 | 30000.0000 |
|       2 |         2 | 52500.0000 |
|       3 |         1 | 45000.0000 |
|    NULL |         1 | 40000.0000 |
+---------+-----------+------------+
+---------+-----------+
| dept_id | total_emp |
+---------+-----------+
|       2 |         2 |
+---------+-----------+
+----------+--------+
| emp_name | salary |
+----------+--------+
| Meera    |  55000 |
| Ravi     |  50000 |
| Kiran    |  45000 |
| Neha     |  40000 |
| Asha     |  30000 |
+----------+--------+
+---------+------------+
| dept_id | avg_salary |
+---------+------------+
|       2 | 52500.0000 |
|       3 | 45000.0000 |
+---------+------------+
```

9- Implement Procedure.(Basic procedure, procedure for Insert, Update and Delete)

INPUT:

```
CREATE TABLE departments (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(50)
);
CREATE TABLE employees (
emp_id INT PRIMARY KEY,
emp_name VARCHAR(50),
```

```
dept_id INT,
salary INT
);
INSERT INTO departments VALUES
(1,'HR'),(2,'IT'),(3,'Finance'),(4,'Marketing');
INSERT INTO employees VALUES
(101,'Asha',1,30000),(102,'Ravi',2,50000),(103,'Meera',2,55000),
(104,'Kiran',3,45000),(105,'Neha',NULL,40000);
DELIMITER //
CREATE PROCEDURE GetAllEmployees()
BEGIN
SELECT * FROM employees;
END //
CREATE PROCEDURE AddEmployee(IN p_id INT, IN p_name VARCHAR(50), IN p_dept
INT, IN p_sal INT)
BEGIN
INSERT INTO employees VALUES (p_id, p_name, p_dept, p_sal);
END //
CREATE PROCEDURE UpdateSalary(IN p_id INT, IN p_sal INT)
BEGIN
UPDATE employees SET salary = p_sal WHERE emp_id = p_id;
END //
CREATE PROCEDURE DeleteEmployee(IN p_id INT)
BEGIN
DELETE FROM employees WHERE emp_id = p_id;
END //
DELIMITER ;
CALL GetAllEmployees();
CALL AddEmployee(106,'Sana',1,35000);
CALL UpdateSalary(106,38000);
CALL DeleteEmployee(106);
CALL GetAllEmployees();
```

OUTPUT:

Output:

```
+--------+----------+---------+--------+
| emp_id | emp_name | dept_id | salary |
+--------+----------+---------+--------+
|    101 | Asha     |       1 |  30000 |
|    102 | Ravi     |       2 |  50000 |
|    103 | Meera    |       2 |  55000 |
|    104 | Kiran    |       3 |  45000 |
|    105 | Neha     |    NULL |  40000 |
+--------+----------+---------+--------+
+--------+----------+---------+--------+
| emp_id | emp_name | dept_id | salary |
+--------+----------+---------+--------+
|    101 | Asha     |       1 |  30000 |
|    102 | Ravi     |       2 |  50000 |
|    103 | Meera    |       2 |  55000 |
|    104 | Kiran    |       3 |  45000 |
|    105 | Neha     |    NULL |  40000 |
+--------+----------+---------+--------+
```

10- Implement Triggers(validation before and after insert, delete and update)

INPUT:

```
CREATE TABLE departments (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(50)
);
CREATE TABLE employees (
emp_id INT PRIMARY KEY,
emp_name VARCHAR(50),
dept_id INT,
salary INT
);
INSERT INTO departments VALUES
(1,'HR'),(2,'IT'),(3,'Finance'),(4,'Marketing');
INSERT INTO employees VALUES
```

```
(101,'Asha',1,30000),(102,'Ravi',2,50000),(103,'Meera',2,55000),
(104,'Kiran',3,45000),(105,'Neha',NULL,40000);
CREATE TABLE emp_audit (
audit_id INT AUTO_INCREMENT PRIMARY KEY,
emp_id INT,
action VARCHAR(20),
action_time DATETIME
);
DELIMITER //
CREATE TRIGGER before_emp_insert BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
IF NEW.salary < 10000 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary must be at least
10000';
END IF;
END //
CREATE TRIGGER after_emp_insert AFTER INSERT ON employees
FOR EACH ROW
BEGIN
INSERT INTO emp_audit (emp_id, action, action_time) VALUES (NEW.emp_id,
'INSERT', NOW());
END //
CREATE TRIGGER before_emp_update BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
IF NEW.salary < OLD.salary THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be
decreased';
END IF;
END //
CREATE TRIGGER after_emp_update AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
INSERT INTO emp_audit (emp_id, action, action_time) VALUES (NEW.emp_id,
'UPDATE', NOW());
END //
CREATE TRIGGER before_emp_delete BEFORE DELETE ON employees
FOR EACH ROW
BEGIN
```

```
IF OLD.salary > 100000 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Cannot delete employee with
salary above 100000';
END IF;
END //
CREATE TRIGGER after_emp_delete AFTER DELETE ON employees
FOR EACH ROW
BEGIN
INSERT INTO emp_audit (emp_id, action, action_time) VALUES (OLD.emp_id,
'DELETE', NOW());
END //
DELIMITER ;
```